

2303a51215

Batch-04

Task 1 — Mutable Default Argument Bug

- Bug: Mutable default argument

```
def add_item(item, items=[]):  
    items.append(item)  
    return items
```

```
print(add_item(1))  
print(add_item(2))
```

Output :

```
[1]  
[1, 2] -Wrong
```

AI Fix :

Use `None` as default and create a new list each call.

Corrected Code

```
def add_item(item, items=None):  
    if items is None:  
        items = []  
  
    items.append(item)  
    return items
```

```
print(add_item(1))  
print(add_item(2))
```

Output:

```
[1]  
[2]
```

Task 2 — Floating-Point Precision Error

Buggy Code:

```
def check_sum():  
    return (0.1 + 0.2) == 0.3
```

```
print(check_sum())
```

Problem:

Floating-point numbers are stored in binary approximation.

$0.1 + 0.2 = 0.30000000000000004$

So comparison fails → **False**.

AI Fix:

Use tolerance comparison via **math.isclose()**.

Corrected Code:

```
import math

def check_sum():
    return math.isclose(0.1 + 0.2, 0.3)

print(check_sum())
```

Output:

True

Task 3 — Recursion Error (Missing Base Case):

Buggy Code:

```
def countdown(n):
    print(n)
    return countdown(n-1)

countdown(5)
```

Problem:

No stopping condition → infinite recursion → stack overflow.

AI Fix:

Add base case when **n == 0**.

Corrected Code:

```
def countdown(n):
    if n == 0:
```

```
    print(0)

    return

print(n)

countdown(n - 1)

countdown(5)
```

Output:

```
5
4
3
2
1
0
```

Task 4 — Dictionary Key Error:

Buggy Code:

```
def get_value():

    data = {"a": 1, "b": 2}

    return data["c"]

print(get_value())
```

Problem:

Key "c" does not exist → `KeyError`.

AI Fix Option 1 — `.get()`:

```
def get_value():

    data = {"a": 1, "b": 2}
```

```
    return data.get("c", "Key not found")

print(get_value())
```

Output:

Key not found

AI Fix Option 2 — Exception Handling:

```
def get_value():

    data = {"a": 1, "b": 2}

    try:

        return data["c"]

    except KeyError:

        return "Key not found"

print(get_value())
```

Task 5 — Infinite Loop:

Buggy Code:

```
def loop_example():

    i = 0

    while i < 5:

        print(i)
```

Problem:

`i` is never incremented → infinite loop.

AI Fix:

Add increment statement.

Corrected Code:

```
def loop_example():
```

```
i = 0

while i < 5:

    print(i)

    i += 1

loop_example()
```

Output:

```
0
1
2
3
4
```

Task 6 — Unpacking Error:

Buggy Code:

```
a, b = (1, 2, 3)
```

Problem:

Too many values to unpack → `ValueError`.

AI Fix Option 1 — Correct Variables:

```
a, b, c = (1, 2, 3)

print(a, b, c)
```

Output:

```
1 2 3
```

AI Fix Option 2 — Ignore Extra Values:

```
a, b, _ = (1, 2, 3)

print(a, b)
```

Output:

12

Task 7 — Mixed Indentation Bug:

Buggy Code:

```
def func():  
  
    x = 5  
  
        y = 10  
  
    return x + y
```

Problem:

Python requires consistent indentation → `IndentationError`.

AI Fix:

Use spaces consistently (PEP 8 → 4 spaces).

Corrected Code:

```
def func():  
  
    x = 5  
  
    y = 10  
  
    return x + y  
  
print(func())
```

Output:

15

Task 8 — Import Error:

```
import maths  
  
print(maths.sqrt(16))
```

Problem:

Module name is incorrect → `ModuleNotFoundError`.

AI Fix:

Correct module name → `math`.

Corrected Code:

```
import math  
  
print(math.sqrt(16))
```

Output:

4.0

AI Debugging Reflection:

AI tools like GitHub Copilot help quickly identify syntax and logical errors.

They suggest fixes such as adding base cases, correcting imports, and improving comparisons.

AI explanations make debugging easier for beginners.

However, developers must still verify AI fixes for correctness.

Systematic debugging combined with AI improves productivity and code reliability.